

Практичні матеріали: Урок 9.3 — Швидкість API, HTTP-кеш та highload

Матеріали для самостійного опрацювання: додати до оптимізації БД/серверного кешу (урок 9.2) **рівень HTTP** — **Cache-Control**, зменшення обсягу JSON, **rate limiting**, винесення важких операцій у **queue jobs**.

Приклади — у фінансовому домені (payments, accounts, reports), за потреби переносимо на інші сценарії.

1. HTTP-кеш: які заголовки куди

Завдання: зрозуміти, які endpoint'и **логічно** кешувати на клієнті/проксі/CDN і як підібрати **Cache-Control** під тип даних.

1.1. Список платежів рахунку — GET /api/v1/accounts/{id}/payments

Персоналізований, часто змінюваний список.

- Мета: трохи розвантажити повторні запити UI (наприклад, «назад/вперед»), але **не** тримати старі дані довго.
- Можливий заголовок:

```
Cache-Control: private, max-age=15
```

- **private** — кеш лише на клієнті (браузер / додаток), не в спільних проксі.
- **max-age=15** — до 15 с повторний запит може взяти локальну копію.

1.2. Довідник валют — GET /api/v1/currencies

Публічний, рідко змінюваний ресурс.

```
Cache-Control: public, max-age=3600
```

- **public** — кеш на будь-якому рівні (браузер, проксі, CDN).
- **max-age=3600** — оновлення не частіше ніж раз на годину.

1.3. Приклад у Laravel: заголовок у відповіді

```
<?php  
  
declare(strict_types=1);  
  
namespace App\Http\Controllers\Api\V1;
```

```

use App\Http\Controllers\Controller;
use App\Http\Resources\Api\V1\CurrencyResource;
use App\Models\Currency;
use Illuminate\Http\JsonResponse;

final class CurrencyController extends Controller
{
    public function index(): JsonResponse
    {
        $currencies = Currency::query()->orderBy('code')->get();

        $response = response()-
>json(CurrencyResource::collection($currencies));

        $response->header('Cache-Control', 'public, max-age=3600');

        return $response;
    }
}

```

Коротко: HTTP-кеш **не** замінює Redis на сервері, а **доповнює**. Для критичних real-time даних (баланс) — зазвичай **no-store** або дуже короткий **private** + чітка логіка оновлення.

2. Обсяг JSON-відповіді

Завдання: для списку платежів **зменшити** payload через Resource/DTO — швидше передача, менше ризику витоку внутрішніх полів.

2.1. «Важкий» варіант (ідея проблеми)

```

{
  "data": [
    {
      "id": 123,
      "account_id": 10,
      "user_id": 5,
      "amount": "250.00",
      "currency": "USD",
      "description": "Оплата рахунку №1001",
      "status": "processed",
      "gateway_payment_id": "gw_abc",
      "gateway_raw_response": "{...}",
      "internal_note": "debug...",
      "created_at": "2026-02-12T10:00:00Z",
      "updated_at": "2026-02-12T10:00:05Z"
    }
  ]
}

```

- `gateway_raw_response`, `internal_note`, зайвий `updated_at` часто **не потрібні** клієнтському UI і збільшують відповідь.

2.2. Обрізаний resource для списку

```
<?php

declare(strict_types=1);

namespace App\Http\Resources\Api\V1;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

/**
 * @property int $id
 * @property int $account_id
 * @property string $amount
 * @property string $currency
 * @property string|null $description
 * @property string $status
 * @property \Carbon\CarbonInterface|null $created_at
 */
final class PaymentListItemResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'account_id' => $this->account_id,
            'amount' => $this->amount,
            'currency' => $this->currency,
            'description' => $this->description,
            'status' => $this->status,
            'created_at' => $this->created_at?->toIso8601String(),
        ];
    }
}
```

Самостійно: переглянути власні відповіді зі списками; виписати поля, потрібні клієнту; оформити Resource/DTO без внутрішніх/зайвих полів.

3. Rate limiting

Завдання: різні ліміти для типових API-методів і для важких операцій (експорт, звіти).

3.1. Базовий ліміт для більшості API

```
<?php
```

```

use App\Http\Controllers\Api\V1\AccountPaymentsController;
use App\Http\Controllers\Api\V1\PaymentsController;
use Illuminate\Support\Facades\Route;

Route::middleware(['auth:sanctum', 'throttle:60,1'])
    ->prefix('api/v1')
    ->group(static function (): void {
        Route::get('accounts/{account}/payments',
[AccountPaymentsController::class, 'index']);
        Route::post('payments', [PaymentsController::class, 'store']);
    });

```

- **throttle:60,1** — 60 запитів на хвилину (на user/token, залежно від конфігурації).

3.2. Жорсткіший ліміт для важких дій

```

<?php

use App\Http\Controllers\Api\V1\ReportsController;

Route::middleware(['auth:sanctum', 'throttle:5,1'])
    ->prefix('api/v1')
    ->group(static function (): void {
        Route::post('reports/account-statement',
[ReportsController::class, 'generate']);
    });

```

- **throttle:5,1** — наприклад, не більше 5 запитів на генерацію за хвилину.
- Ліміти можна виносити в окремі групи / по ролях. Фронтенд має коректно обробляти **429** (повідомлення, повтор пізніше).

4. Queue jobs для важких операцій

Завдання: винести важку операцію (експорт виписки) з «чекаємо PDF/CSV в одному HTTP-запиті» у модель «**прийняли задачу — результат згодом**».

4.1. Job-скелет: експорт виписки (Laravel)

```

<?php

declare(strict_types=1);

namespace App\Jobs;

use App\Models\Account;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;

```

```

use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

final class ExportAccountStatementJob implements ShouldQueue
{
    use Dispatchable;
    use InteractsWithQueue;
    use Queueable;
    use SerializesModels;

    public function __construct(
        private readonly int $accountId,
        private readonly string $periodFrom,
        private readonly string $periodTo,
        private readonly int $requestedByUserId
    ) {
    }

    public function handle(): void
    {
        /** @var Account $account */
        $account = Account::query()->findOrFail($this->accountId);

        // Зібрати операції, згенерувати CSV/PDF, зберегти в сховище,
        // за потреби – нотифікація користувачу.
    }
}

```

4.2. Контролер: поставив job, швидко відповів

```

<?php

declare(strict_types=1);

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Jobs\ExportAccountStatementJob;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

final class ReportsController extends Controller
{
    public function generateAccountStatement(Request $request):
    JsonResponse
    {
        $validated = $request->validate([
            'account_id' => ['required', 'integer'],
            'from' => ['required', 'date'],
            'to' => ['required', 'date', 'after_or_equal:from'],
        ]);
    }
}

```

```

        ExportAccountStatementJob::dispatch(
            (int) $validated['account_id'],
            (string) $validated['from'],
            (string) $validated['to'],
            (int) $request->user()->id
        );

        return response()->json([
            'status' => 'accepted',
            'message' => 'Report generation started.',
        ], 202);
    }
}

```

- **202** — API не чекає на генерацію; воркер обробляє job у фоні.
- Наступний крок у реальному продукті: endpoint статусу або сповіщення про готовність файлу.

5. Міні-чеклист для власного API

Завдання: зафіксувати поточний стан API і **2–3** кроки з найбільшим ефектом.

Для ключових endpoint'ів (платежі, рахунки, звіти) варто відповісти:

- пагінація?
- серверний кеш (Redis) там, де це безпечно?
- HTTP-кеш для публічних/рідко мінливих даних?
- rate limiting?
- важкі операції в **jobs**?
- індекси, без N+1, виміри (профайлери), логи / Sentry під навантаженням?

Самостійно: обрати 2–3 пункти, які реально внедрити за 1–2 тижні, і визначити, **як** вимірювати ефект (час відповіді, помилки, навантаження на БД).

6. Середовище (Docker)

- HTTP-кеш і rate limiting **зазвичай не** вимагають змін у **docker-compose** — рівень застосунку (і зовнішнього проксі/CDN).
- Для **queue** у стеку має бути **воркер** (як у модулі 5).
- Профілювання «під навантаженням» — стабільне середовище та моніторинг (модуль 8).

Підсумок

- Архітектура, тести, логи, кеш, БД і **HTTP-рівень** складають одну картинку **production-ready** API.
- Невеликі кроки — HTTP-кеш для довідників, обрізаний JSON, ліміти, jobs для важких задач — дають **вимірювану** вигід без «переписати все».
- Це знижує ризик, коли навантаження росте: є що вмикати і що міряти.